

SOC/L1 - 7.6. Snort

Task 1 - Introduction

SNORT is an open-source, rule-based Network Intrusion Detection and Prevention System (NIDS/NIPS). Snort IPS uses a series of rules that help define malicious network activity and uses those rules to find packets that match against them and generate alerts for users."

Task 2 - Interactive Material and VM

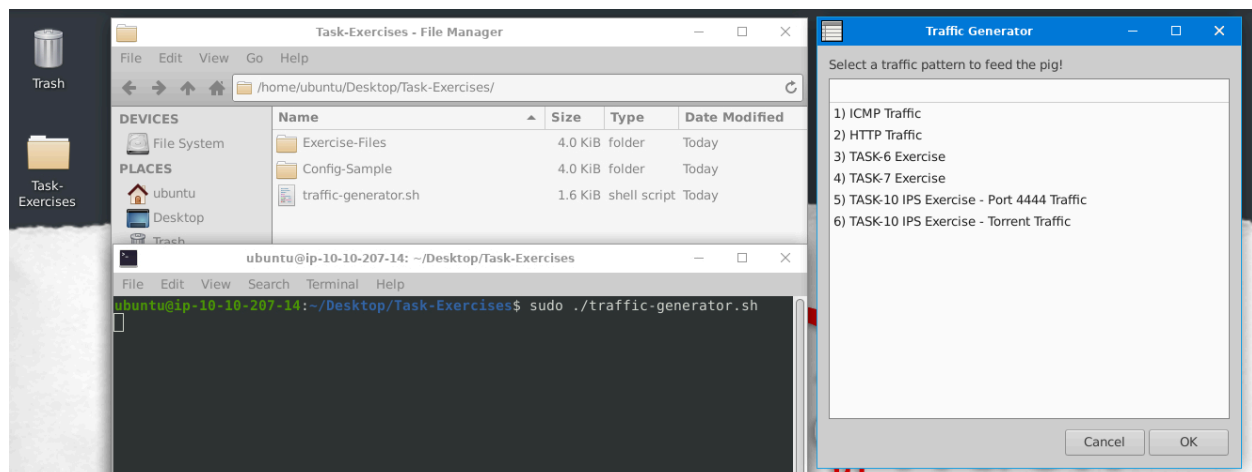
There are two sub-folders available:

- **Config-Sample** : Sample configuration and rule files. Snort's original base files are located under `/etc/snort` folder.
- **Exercise-Files** : There are separate folders for each task.

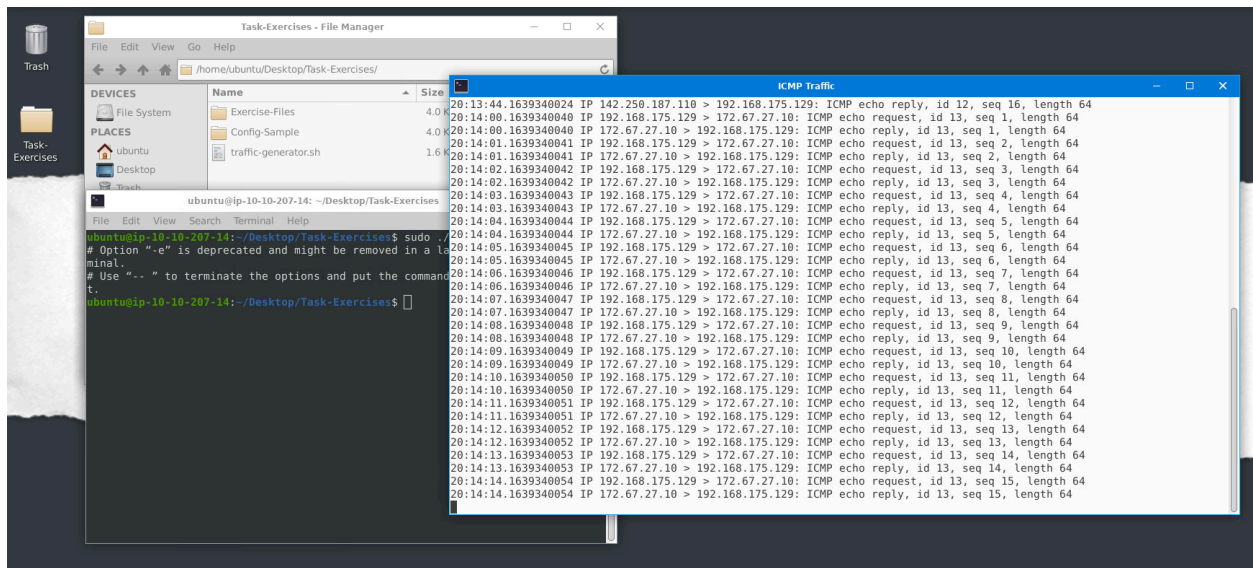
Traffic Generator: there is a script `traffic-generator.sh` for you to generate traffic to your Snort interface. You will use this script to trigger traffic to the Snort interface. Once you run the script, it will ask you to choose the exercise type and then automatically open another terminal to show you the output of the selected action. Run the "traffic generator.sh" file by executing it as sudo.

```
user@ubuntu$ sudo ./traffic-generator.sh
```

General desktop overview. Traffic generator script in action.

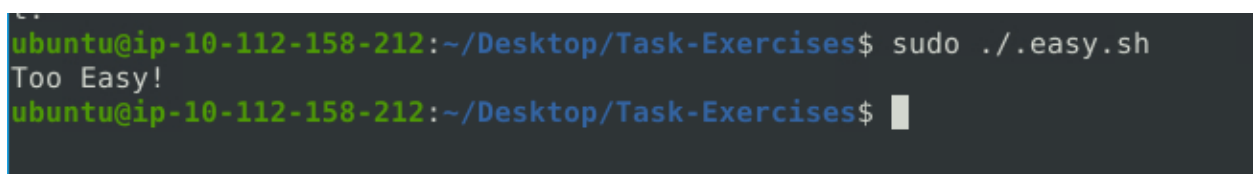


Once you choose an action, the menu disappears, and a terminal instance opens to show you its output.



Answer the questions below

Navigate to the Task-Exercises folder and run the command `./easy.sh` and write the output. Too Easy! (executed the shell script named `easy.sh` located in the current directory)



Task 3 - Introduction to IDS/IPS

Intrusion Detection System (IDS): IDS is a passive monitoring solution for detecting possible malicious activities/patterns, abnormal incidents, and policy violations. It generates alerts for each suspicious event. There are two main types of IDS systems:

- **Network Intrusion Detection System (NIDS):** NIDS monitors the traffic flow from various areas of the network. The aim is to investigate the traffic on the entire subnet. If a signature is identified, an alert is created.

- **Host-based Intrusion Detection System (HIDS):** HIDS monitors the traffic flow from a single endpoint device. Its aim is to investigate the traffic on that device. If a signature is identified, an alert is created.

Intrusion Prevention System (IPS): IPS is an active protecting solution for preventing possible malicious activities/patterns, abnormal incidents, and policy violations. It is responsible for stopping/preventing/terminating the suspicious event as soon as it is detected. There are four main types of IPS systems;

- **Network Intrusion Prevention System (NIPS):** NIPS monitors the traffic flow from various areas of the network. The aim is to protect the traffic on the entire subnet. If a signature is identified, the connection is terminated.
- **Behaviour-based Intrusion Prevention System (Network Behaviour Analysis - NBA):** Behaviour-based systems monitor the traffic flow from various areas of the network. The aim is to protect the traffic on the entire subnet. If an anomaly is identified, the connection is terminated.
- **Wireless Intrusion Prevention System (WIPS):** WIPS monitors the traffic flow from a wireless network. Its aim is to protect wireless traffic and stop possible attacks launched from there. If a signature is identified, the connection is terminated.
- **Host-based Intrusion Prevention System (HIPS):** HIPS actively protects the traffic flow from a single endpoint device. The aim is to investigate the traffic on a particular device. If a signature is identified, the connection is terminated.

Detection/Prevention Techniques

There are three main detection and prevention techniques used in IDS and IPS solutions;

Technique	Approach
Signature-Based	This technique relies on rules that identify the specific patterns of the known malicious behaviour. This model helps detect known threats.
Behaviour-Based	This technique identifies new threats with new patterns that pass through signatures. The model compares known/normal behaviours with unknown/abnormal behaviours, helping detect previously unknown or new threats.

Policy-Based	This technique compares detected activities with system configuration and security policies. This model helps detect policy violations.
---------------------	---

SNORT is an open-source, rule-based Network Intrusion Detection and Prevention System (NIDS/NIPS). Snort has the following capabilities:

- Live traffic analysis
- Attack and probe detection
- Packet logging
- Protocol analysis
- Real-time alerting
- Modules & plugins
- Pre-processors
- Cross-platform support! (Linux & Windows)

Snort Modes: Snort has the following three main use modes:

- **Sniffer Mode:** Read and prompt IP packets in the console application.
- **Packet Logger Mode:** Log all IP packets (inbound and outbound) that visit the network.
- **NIDS (Network Intrusion Detection System) and NIPS (Network Intrusion Prevention System) Modes:** Log/drop the packets deemed malicious according to the user-defined rules.

Answer the questions below

- | Which IDS or IPS type can help you stop the threats on a local machine? HIPS
- | Which IDS or IPS type can help you detect threats on a local network? NIDS
- | Which IDS or IPS type can help you detect the threats on a local machine? HIDS
- | Which IDS or IPS type can help you stop the threats on a local network? NIPS
- | Which described solution works by detecting anomalies in the network? NBA

According to the official description of the snort, what kind of NIPS is it? full-blown

NBA training period is also known as ... baselining

Task 4 - First Interaction with Snort

The First Interaction with Snort

```
version check: snort -V
```

```
ubuntu@ip-10-112-158-212:~/Desktop/Task-Exercises$ snort -V
''_~
o" )~
' ' '

-*> Snort! <*-
Version 2.9.7.0 GRE (Build 149)
By Martin Roesch & The Snort Team: http://www.snort.org/contact#team
Copyright (C) 2014 Cisco and/or its affiliates. All rights reserved.
Copyright (C) 1998-2013 Sourcefire, Inc., et al.
Using libpcap version 1.9.1 (with TPACKET_V3)
Using PCRE version: 8.39 2016-06-14
Using ZLIB version: 1.2.11
```

Here, "-T" is used to test the configuration, and "-c" identifies configuration file (**snort.conf**). Note that it is possible to use an additional configuration file by pointing to it with "-c".

Configuration check: `sudo snort -c /etc/snort/snort.conf -T`

Parameter	Description
-V / --version	This parameter provides information about your instance version.
-c	Identifying the configuration file
-T	Snort's self-test parameter allows you to test your setup with this parameter.
-q	Quiet mode prevents Snort from displaying the default banner and initial information about your setup.

Answer the questions below

Run the Snort instance and check the build number. 149

```

ubuntu@ip-10-112-158-212:~/Desktop/Task-Exercises$ snort -V
    ,,,
    o" )~  -*> Snort! <*-
    '""   Version 2.9.7.0 GRE (Build 149)
          By Martin Roesch & The Snort Team: http://www.snort.org/c
          Copyright (C) 2014 Cisco and/or its affiliates. All right
          Copyright (C) 1998-2013 Sourcefire, Inc., et al.
          Using libpcap version 1.9.1 (with TPACKET_V3)
          Using PCRE version: 8.39 2016-06-14
          Using ZLIB version: 1.2.11

```

Test the current instance with "/etc/snort/snort.conf" file and check how many rules are loaded with the current build? 4151

run the query: `sudo snort -c /etc/snort/snort.conf -T`

```

4151 Snort rules read
    3477 detection rules
    0 decoder rules
    0 preprocessor rules
3477 Option Chains linked into 271 Chain Headers
0 Dynamic rules

```

Test the current instance with "/etc/snort/snortv2.conf" file and check how many rules are loaded with the current build? 1

run the query: `sudo snort -c /etc/snort/snort.confv2 -T`

```

+++++
Initializing rule chains...
1 Snort rules read
    1 detection rules
    0 decoder rules
    0 preprocessor rules
1 Option Chains linked into 1 Chain Headers
0 Dynamic rules
+++++

```

Task 5 - Operation Mode 1: Sniffer Mode

Snort sniffing mode captures and displays live network packets for analysis without applying detection rules or generating alerts. Sniffer mode parameters are explained in the table below;

Parameter	Description
-v	Verbose. Display the TCP/IP output in the console.
-d	Display the packet data (payload).
-e	Display the link-layer (TCP/IP/UDP/ICMP) headers.
-X	Display the full packet details in HEX.
-i	This parameter helps define a specific network interface to listen to or sniff. Once you have multiple interfaces, you can choose a specific interface to sniff

Sniffing with parameter "-i" : Start the Snort instance in **verbose mode (-v)** and **use the interface (-i)** "eth0"; `sudo snort -v -i eth0`

Sniffing with parameter "-v" : Start the Snort instance in **verbose mode (-v)**; `sudo snort -v`

Sniffing with parameter "-d" : Start the Snort instance in **dumping packet data mode (-d)**; `sudo snort -d`

Sniffing with parameter "-de" : Start the Snort instance in **dump (-d)** and **link-layer header grabbing (-e)** mode; `snort -d -e`

Sniffing with parameter "-X" : Start the Snort instance in **full packet dump mode (-X)**; `sudo snort -X`

Task 6 - Operation Mode 2: Packet Logger Mode

Snort Packet Logger Mode captures network packets and stores them in log files for later analysis, making it ideal for forensic investigations and incident response. Packet logger parameters are explained in the table below;

Parameter	Description
-l	Logger mode, target log, and alert output directory. Default output folder is /var/log/snort The default action is to dump as tcpdump format in /var/log/snort
-K ASCII	Log packets in ASCII format.
-r	Reading option: Review the logged events in Snort.
-n	Specify the number of packets to be processed or read. Snort will stop after reading the specified number of packets.

Logging with parameter "-l" : Start the Snort instance in packet logger mode; `sudo snort -dev -l .`

Then start ICMP/HTTP traffic with the traffic-generator script.

Once the traffic is generated, Snort will start showing the packets and log them in the target directory. You can configure the default output directory in Snort.config file. However, you can use the "-l" parameter to set a target directory. Identifying the default log directory is useful for continuous monitoring operations, and the "-l" parameter is much more useful for testing purposes.

- **d** → Logs the application-layer payload (packet data).
- **e** → Includes the Ethernet (Layer 2) header in the output.
- **v** → Displays packet headers and information on the screen.

`sudo snort -dev -l .` runs Snort with root privileges, captures network traffic, displays detailed packet information, and logs the captured packets in the current directory.

```
user@ubuntu$ ls.  
snort.log.1638459842
```

As you can see, it is a single all-in-one log file. It is a binary/tcpdump format log. This is what it looks like in the folder view. `-l .` tells Snort to save the logs in the current working directory. This is convenient for labs, testing, or when you don't want to write to the default system log directory

Name	Size
 snort.log.1638459842	6.3 kB

Logging with parameter "-K ASCII"

Start the Snort instance in packet logger mode; `sudo snort -dev -K ASCII`

Now run the traffic-generator script as sudo and start ICMP/HTTP traffic.

Checking the log file

```
user@ubuntu$ ls .
```

```
142.250.187.110 192.168.175.129 snort.log.1638459842
```

This is what it looks like in the folder view.

Name	Size
 142.250.187.110	1 item
 192.168.175.129	5 items
 snort.log.1638459842	6.3 kB

The logs created with the "-K ASCII" parameter are entirely different. There are two folders with IP address names.

```
user@ubuntu$ ls ./192.168.175.129/
```

```
ICMP_ECHO  UDP:36648-53  UDP:40757-53  UDP:47404-53  UDP:5062  
4-123
```

Reading generated logs with parameter "-r" : Start the Snort instance in packet reader mode;

```
sudo snort -r snort.log.1638459842
```

Note that Snort can read and handle binary log output (tcpdump and Wireshark can also handle this log format). However, if you create logs with the "-K ASCII" parameter, Snort will not read them. As you can see in the output above, Snort read and displayed the log file, just as it does in sniffer mode.

`sudo tcpdump -r snort.log.1638459842 -ntc 10` reads the first 10 packets from a Snort capture file and displays them using raw IP addresses and port numbers, without timestamps or name resolution.

- `tcpdump` → A command-line packet capture and analysis tool.

- `r snort.log.1638459842` → Reads packets from the specified Snort log (PCAP) file instead of capturing live traffic.
- `n` → Does **not** resolve IP addresses to hostnames or port numbers to service names. Displays raw IP addresses and port numbers, making the output faster and clearer.
- `t` → Omits the timestamp from each packet in the output.
- `c 10` → Stops after displaying the **first 10 packets**.

Answer the questions below

Investigate the traffic with the default configuration file with ASCII mode.

run the query: `sudo snort -dev -K ASCII -l`

Execute the traffic generator script and choose "TASK-6 Exercise". Wait until the traffic ends, then stop the Snort instance. Now analyse the output summary and answer the question.

run the query: `sudo ./traffic-generator.sh`

Now, you should have the logs in the current directory. Navigate to folder "145.254.160.237". What is the source port used to connect port 53? 3009

```
Snort exiting
ubuntu@ip-10-112-158-212:~/Desktop/Task-Exercises$ ls
10.112.158.212  172.67.27.10  PACKET_NONIP  snort.log.1785152712
10.112.87.53    192.168.175.129  fe80::3:cdff:fe67:46d7  traffic-generator.sh
142.250.187.110  Config-Sample  snort.log.1785151557
145.254.160.237  Exercise-Files  snort.log.1785152393
ubuntu@ip-10-112-158-212:~/Desktop/Task-Exercises$ sudo ls ./145.254.160.237
TCP:3371-80  TCP:3372-80  UDP:3009-53
ubuntu@ip-10-112-158-212:~/Desktop/Task-Exercises$
```

Use snort.log.1640048004. Read the snort.log file with Snort; what is the IP ID of the 10th packet? 49313

run the query: `snort -r snort.log.1640048004 -n 10`

```

=====
WARNING: No preprocessors configured for policy 0.
05/13-10:17:09.754737 65.208.228.223:80 -> 145.254.160.237:3372
TCP TTL:47 TOS:0x0 ID:49313 IpLen:20 DgmLen:1420 DF
***A*** Seq: 0x114C6C54 Ack: 0x38AFFFF3 Win: 0x1920 TcpLen: 20
=====

```

Read the "snort.log.1640048004" file with Snort; what is the referer of the 4th packet?
<http://www.ethereal.com/development.html>

run the query: `snort -r snort.log.1640048004 -n 4 -X`

```

0x01B0: 71 3D 30 2E 37 0D 0A 4B 65 65 70 2D 41 6C 69 76  q=0.7..Keep-Alive
0x01C0: 65 3A 20 33 30 30 0D 0A 43 6F 6E 6E 65 63 74 69  e: 300..Connecti
0x01D0: 6F 6E 3A 20 6B 65 65 70 2D 61 6C 69 76 65 0D 0A  on: keep-alive..
0x01E0: 52 65 66 65 72 65 72 3A 20 68 74 74 70 3A 2F 2F  Referer: http://
0x01F0: 77 77 77 2E 65 74 68 65 72 65 61 6C 2E 63 6F 6D  www.ethereal.com
0x0200: 2F 64 65 76 65 6C 6F 70 6D 65 6E 74 2E 68 74 6D  /development.htm
0x0210: 6C 0D 0A 0D 0A 1....

```

Read the "snort.log.1640048004" file with Snort; what is the Ack number of the 8th packet? 0x38AFFFF3

run the query: `snort -r snort.log.1640048004 -n 8`

```

=====
WARNING: No preprocessors configured for policy 0.
05/13-10:17:09.123830 65.208.228.223:80 -> 145.254.160.237:3372
TCP TTL:47 TOS:0x0 ID:49312 IpLen:20 DgmLen:1420 DF
***A*** Seq: 0x114C66F0 Ack: 0x38AFFFF3 Win: 0x1920 TcpLen: 20
=====

```

Read the "snort.log.1640048004" file with Snort; what is the number of the "TCP port 80" packets? 41

run the query: `sudo snort -r snort.log.1640048004 'tcp port 80'`

```

    UDP Disc:          0 ( 0.000%)
    ICMP Disc:         0 ( 0.000%)
  All Discard:         0 ( 0.000%)
    Other:             0 ( 0.000%)
  Bad Chk Sum:         0 ( 0.000%)
    Bad TTL:           0 ( 0.000%)
    S5 G 1:            0 ( 0.000%)
    S5 G 2:            0 ( 0.000%)
    Total:             41

```

Task 7 - Operation Mode 3: IDS/IPS

Snort in IDS/IPS Mode

Parameter	Description
-c	Defining the configuration file.
-T	Testing the configuration file.
-N	Disable logging.
-D	Background mode.
-A	Alert modes;Full: Full alert mode, providing all possible information about the alert. This mode is also the default; once you use -A and don't specify a mode, Snort defaults to this mode. Fast mode displays the alert message, timestamp, source and destination IP addresses, along with port numbers. Console: Provides fast style alerts on the console screen. cmg: CMG style, basic header details with payload in hex and text format. None: Disabling alerting.

Once you start running IDS/IPS mode, you need to use rules.

```
alert icmp any any <> any any (msg: "ICMP Packet Found"; sid: 100001; rev:1;)
```

This rule is located in "/etc/snort/rules/local.rules".

IDS/IPS mode with parameter "-c and -T" : `Sudo snort -c /etc/snort/snort.conf -T`

This command will check your configuration file and prompt you if there is any misconfiguration in your current settings.

IDS/IPS mode with parameter "-N" : `sudo snort -c /etc/snort/snort.conf -N` This command will disable logging mode. The rest of the other functions will still be available (if activated).

IDS/IPS mode with parameter "-D" : `sudo snort -c /etc/snort/snort.conf -D` Snort will begin processing the packets and accomplish the given task with additional parameters.

The command-line output will provide the requested information along with the specified parameters. So, if you activate **verbosity (-v)** or **full packet dump (-X)** with **packet logger mode (-l)**, you will still have the logs in the logs folder, but there will be no output in the console. Once you start the background mode and want to check the corresponding process, you can easily use the "ps" command as shown below;

```
user@ubuntu$ ps -ef | grep snort

root          2898      1706  0 05:53 ?           00:00:00 snort -c
/etc/snort/snort.conf -D
```

IDS/IPS mode with parameter "-A"

Note that Snort offers several alert modes.

- **console:** Provides fast style alerts on the console screen.
- **cmg:** Provides basic header details with payload in hex and text format.
- **full:** Full alert mode, providing all possible information about the alert.
- **fast:** Fast mode, shows the alert message, timestamp, source, and destination ip along with port numbers.
- **none:** Disabling alerting.

IDS/IPS mode with parameter "-A console" : Console mode provides fast style alerts on the console screen. Start the Snort instance in **console** alert mode (`-A console`) with the following command: `sudo snort -c /etc/snort/snort.conf -A console`

IDS/IPS mode with parameter "-A cmg" : CMG mode provides basic header details, including the payload, in both hex and text formats. Start the Snort instance in cmg alert mode (`-A cmg`) with the following command: `sudo snort -c /etc/snort/snort.conf -A cmg`

IDS/IPS mode with parameter "-A fast" : Fast mode provides alert messages, timestamps, and source and destination IP addresses. **Remember, there is no**

console output in this mode. Start the Snort instance in fast alert mode (-A fast) with the following command: `sudo snort -c /etc/snort/snort.conf -A fast`

IDS/IPS mode with parameter "-A full" : Full alert mode provides all possible information about the alert. **Remember, there is no console output in this mode.** Start the Snort instance in full alert mode (-A full) with the following command: `sudo snort -c /etc/snort/snort.conf -A full`

IDS/IPS mode with parameter "-A none" : Disable alerting. This mode doesn't create the alert file. However, it still logs the traffic and creates a log file in binary dump format. Remember, there is no console output in this mode. Start the Snort instance in non-alert mode (-A none) with the following command: `sudo snort -c /etc/snort/snort.conf -A none`

Answer the questions below

Investigate the traffic with the default configuration file.

run the query: `sudo snort -c /etc/snort/snort.conf -A full -l .`

Execute the traffic generator script and choose **"TASK-7 Exercise"**. Wait until the traffic stops, then stop the Snort instance. Now analyse the output summary and answer the question.

run the query: `sudo ./traffic-generator.sh`

What is the number of the detected HTTP GET methods? 2

```

P Inspect - encodings (Note: stream-reassembled p
POST methods:                                0
GET methods:                                2
HTTP Request Headers extracted:              2
HTTP Request Cookies extracted:              0
Post parameters extracted:                  0
HTTP response Headers extracted:             3
HTTP Response Cookies extracted:             0
Unicode:                                    0
Double unicode:                             0
Non-ASCII representable:                    0
Directory traversals:                      0
Extra slashes ("//"):                       1
Self-referencing paths ("./"):              0
HTTP Response Gzip packets extracted:        1
Gzip Compressed Data Processed:             1272.00
Gzip Decompressed Data Processed:           3608.00
Total packets processed:                    1405

```

You can practice the rest of the parameters by using the traffic-generator script. Co

Task 8 - Operation Mode 4: PCAP Investigation

PCAP mode parameters are explained in the table below;

Parameter	Description
-r / --pcap-single=	Read a single pcap
--pcap-list=""	Read pcaps provided in command (space separated).
--pcap-show	Show pcap name on console during processing.

Investigating single PCAP with parameter "-r" : For test purposes, you can still test the default reading option with pcap by using the following command `snort -r icmp-test.pcap`

Investigating multiple PCAPs with parameter "--pcaplist": `sudo snort -c /etc/snort/snort.conf -q --pcap-list="icmp-test.pcap http2.pcap" -A console -n 10`

Investigating multiple PCAPs with parameter "--pcapshow" : `sudo snort -c /etc/snort/snort.conf -q --pcap-list="icmp-test.pcap http2.pcap" -A console --pcap-show`

Answer the questions below

Investigate the mx-1.pcap file with the default configuration file.

run the query `sudo snort -c /etc/snort/snort.conf -A full -l . -r mx-1.pcap`

What is the number of the generated alerts? 170

```
=====
Action Stats:
  Alerts:          170 (147.826%)
  Logged:          170 (147.826%)
  Passed:           0 (  0.000%)
Limits:
  Match:           0
```

Keep reading the output. How many TCP Segments are Queued? 18

```
TCP Overlaps: 0
TCP Segments Queued: 18
TCP Segments Released: 18
TCP Rebuilt Packets: 5
TCP Segments Used: 18
TCP Discards: 1
TCP Gaps: 0
```

Keep reading the output. How many "HTTP response headers" were extracted?
3

Investigate the mx-1.pcap file with the second configuration file.

run the query: `sudo snort -c /etc/snort/snortv2.conf -A full -l . -r mx-1.pcap`

What is the number of the generated alerts? 68

```
=====
Action Stats:
  Alerts:          68 ( 59.130%)
  Logged:          68 ( 59.130%)
  Passed:           0 (  0.000%)
```

Investigate the mx-2.pcap file with the default configuration file.

run the query: `sudo snort -c /etc/snort/snort.conf -A full -l . -r mx-2.pcap`

What is the number of the generated alerts? 340

Keep reading the output. What is the number of the detected TCP packets? 82

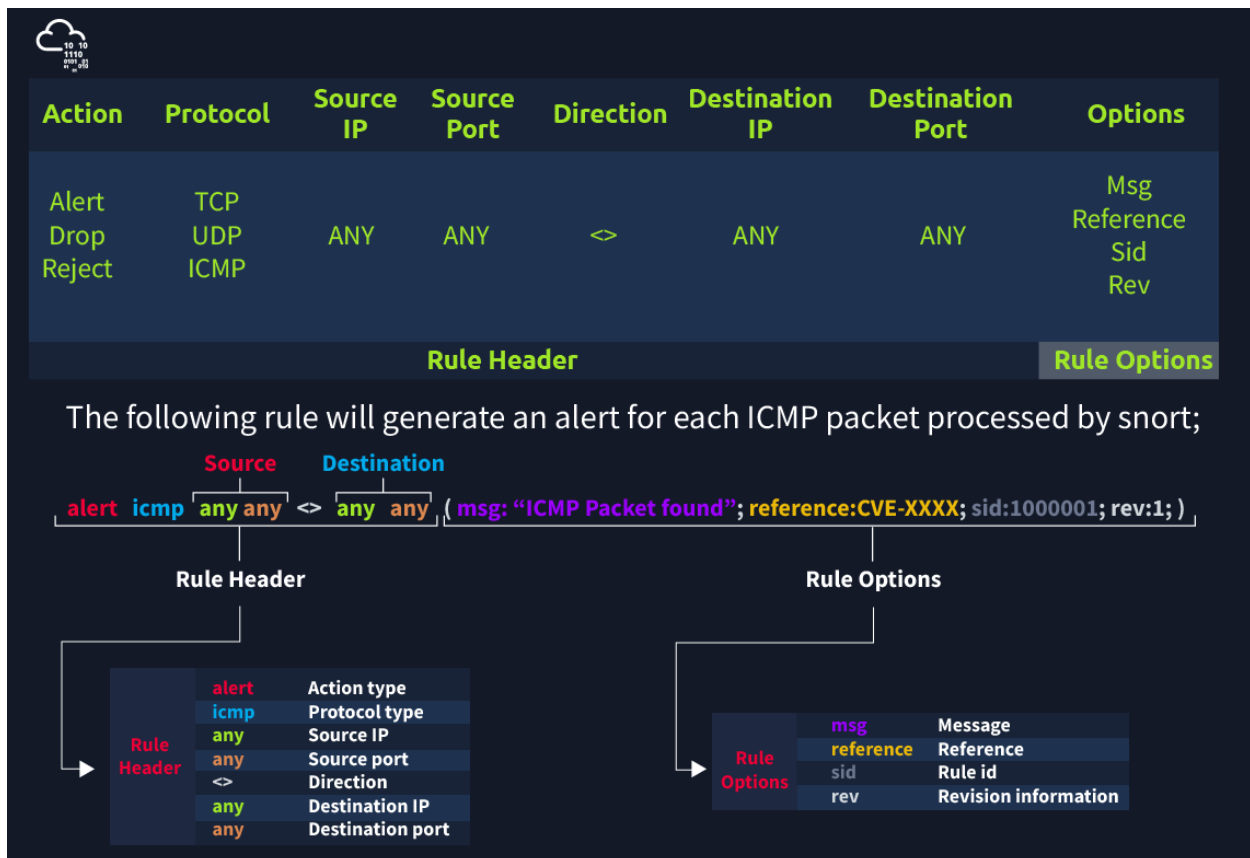
Investigate the mx-2.pcap and mx-3.pcap files with the default configuration file.

run the query: `sudo snort -c /etc/snort/snort.conf -A full -l . --pcap-list="mx-2.pcap mx-3.pcap"`

What is the number of the generated alerts? 1020

Task 9 - Snort Rule Structure

the Snort rule is shown below;



Action	There are several actions for rules. Ensure you understand the functionality and test it thoroughly before creating rules for live systems. The following are the most common actions: • alert: Generate an alert and log the packet. • log: Log the packet. • drop: Block and log the packet. • reject: Block the packet, log it, and terminate the packet session.
Protocol	The protocol parameter identifies the type of protocol that was filtered for the rule. Note that Snort2 supports only four protocol filters in the rules

(IP, TCP, UDP, and ICMP). However, you can detect the application flows using port numbers and options. For instance, if you want to detect FTP traffic, you cannot use the FTP keyword in the protocol field; instead, you can filter FTP traffic by investigating TCP traffic on port 21.

IP and Port Numbers

IP Filtering	alert icmp 192.168.1.56 any <> any any (msg: "ICMP Packet From "; sid: 100001; rev:1;)This rule will create an alert for each ICMP packet originating from the 192.168.1.56 IP address.
Filter an IP range	alert icmp 192.168.1.0/24 any <> any any (msg: "ICMP Packet Found"; sid: 100001; rev:1;)This rule will create an alert for each ICMP packet originating from the 192.168.1.0/24 subnet.
Filter multiple IP ranges	alert icmp [192.168.1.0/24, 10.1.1.0/24] any <> any any (msg: "ICMP Packet Found"; sid: 100001; rev:1;)This rule will create an alert for each ICMP packet originating from the 192.168.1.0/24 and 10.1.1.0/24 subnets.
Exclude IP addresses/ranges.	The "negation operator" is used to exclude specific addresses and ports. The negation operator is indicated with "!".alert icmp !192.168.1.0/24 any <> any any (msg: "ICMP Packet Found"; sid: 100001; rev:1;)This rule will create an alert for each ICMP packet not originating from the 192.168.1.0/24 subnet.
Port Filtering	alert tcp any any <> any 21 (msg: "FTP Port 21 Command Activity Detected"; sid: 100001; rev:1;)This rule will create an alert for each TCP packet sent to port 21.
Exclude a specific port	alert tcp any any <> any !21 (msg: "Traffic Activity Without FTP Port 21 Command Channel"; sid: 100001; rev:1;)This rule will create an alert for each TCP packet not sent to port 21.
Filter a port range (Type 1)	alert tcp any any <> any 1:1024 (msg: "TCP 1-1024 System Port Activity"; sid: 100001; rev:1;)This rule will create an alert for each TCP packet sent to ports between 1 and 1024.
Filter a port range (Type 2)	alert tcp any any <> any :1024 (msg: "TCP 0-1024 System Port Activity"; sid: 100001; rev:1;)This rule will create an alert for each TCP packet sent to ports 1 through 1024.
Filter a port range (Type 3)	alert tcp any any <> any 1025: (msg: "TCP Non-System Port Activity"; sid: 100001; rev:1;)This rule will create an alert for each TCP packet sent to a source port that is higher than or equal to 1025.

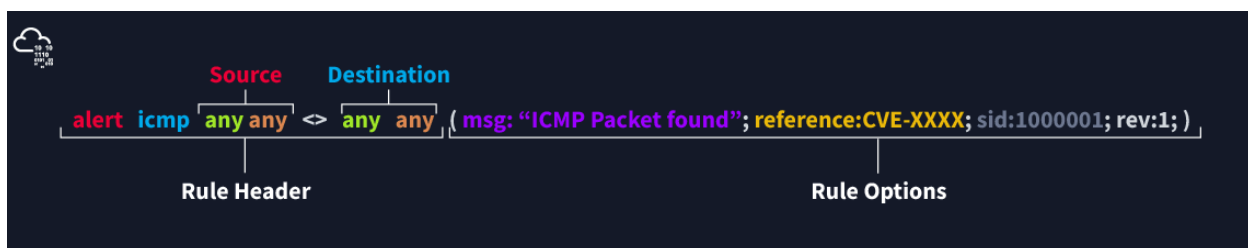
Filter a port range (Type 4)	<pre>alert tcp any any <> any [21,23] (msg: "FTP and Telnet Port 21-23 Activity Detected"; sid: 100001; rev:1;)</pre> This rule will create an alert for each TCP packet sent to ports 21 and 23.
-------------------------------------	--

Direction

The direction operator indicates the traffic flow to be filtered by Snort. The left side of the rule shows the source, and the right side shows the destination.

- > Source to destination flow.
- <> Bidirectional flow

Note that **there is no "<" operator in Snort.**



There are three main rule options in Snort.

- General Rule Options - Fundamental rule options for Snort.
- Payload Rule Options - Rule options that help to investigate the payload data. These options are helpful to detect specific payload patterns.
- Non-Payload Rule Options - Rule options that focus on non-payload data. These options will help create specific patterns and identify network issues.

General Rule Options

Msg	The message field is a basic prompt and quick identifier of the rule. Once the rule is triggered, the message field will appear in the console or log. Usually, the message part is a one-liner that summarises the event.
Sid	Snort rule IDs (SIDs) come with a predefined scope, and each rule must have a SID in the proper format. There are three different scopes for SIDs shown below. • <100: Reserved rules • 100-999,999: Rules came with the build. • >=1,000,000: Rules created by user.

	Briefly, the rules we will create should have a side greater than 100.000.000. Another important point is that SIDs should not overlap, and each ID must be unique.
Reference	Each rule can have additional information or a reference to explain the purpose of the rule or threat pattern. That could be a Common Vulnerabilities and Exposures (CVE) ID or external data. Having references for the rules will always help analysts during the alert and incident investigation.
Rev	Snort rules can be modified and updated for performance and efficiency issues. The revision option enables analysts to access revision information for each rule. Therefore, it will be easy to understand rule improvements. Each rule has its unique rev number, and there is no auto-backup feature on the rule history. Analysts should keep their own rule history. The 'Rev' option is only an indicator of how many times the rule has undergone revisions. alert icmp any any <> any any (msg: "ICMP Packet Found"; sid: 100001; reference:cve,CVE-XXXX; rev:1;)

Payload Detection Rule Options

Content	Payload data. It matches specific payload data by ASCII, HEX, or both. This option can be used multiple times within a single rule. However, the more specific pattern match features you create, the more time it takes to investigate a packet. The following rules will create an alert for each HTTP packet containing the keyword "GET". This rule option is case sensitive! • ASCII mode - alert tcp any any <> any 80 (msg: "GET Request Found"; content:"GET"; sid: 100001; rev:1;) • HEX mode - alert tcp any any <> any 80 (msg: "GET Request Found"; content:" 47 45 54 "; sid: 100001; rev:1;)
Nocase	Disabling case sensitivity. Used for enhancing the content searches.alert tcp any any <> any 80 (msg: "GET Request Found"; content:"GET"; nocase; sid: 100001; rev:1;)
Fast_pattern	Prioritise content search to speed up the payload search operation. By default, Snort uses the largest content and evaluates it against the rules. The "fast_pattern" option helps you select the initial packet match with a specific value for further investigation. This option always works case-insensitively and can be used once per rule. Note that this option is required when using multiple "content" options. The following rule has two content options, and the fast_pattern option

	instructs Snort to use the first content option (in this case, "GET") for the initial packet match. alert tcp any any <> any 80 (msg: "GET Request Found"; content:"GET"; fast_pattern; content:"www"; sid:100001; rev:1;)
--	--

Non-Payload Detection Rule Options

ID	Filtering the IP ID field. alert tcp any any <> any any (msg: "ID TEST"; id:123456; sid: 100001; rev:1;)
Flags	Filtering the TCP flags. • F - FIN • S - SYN • R - RST • P - PSH • A - ACK • U - URG alert tcp any any <> any any (msg: "FLAG TEST"; flags:S; sid: 100001; rev:1;)
Dsize	Filtering the packet payload size. • dsize:min<>max; • dsize:>100 • dsize:<100 alert ip any any <> any any (msg: "SEQ TEST"; dsize:100<>300; sid: 100001; rev:1;)
Sameip	Filtering the source and destination IP addresses for duplication. alert ip any any <> any any (msg: "SAME-IP TEST"; sameip; sid: 100001; rev:1;)

```
user@ubuntu$ sudo gedit /etc/snort/rules/local.rules
```

Open ▾ 

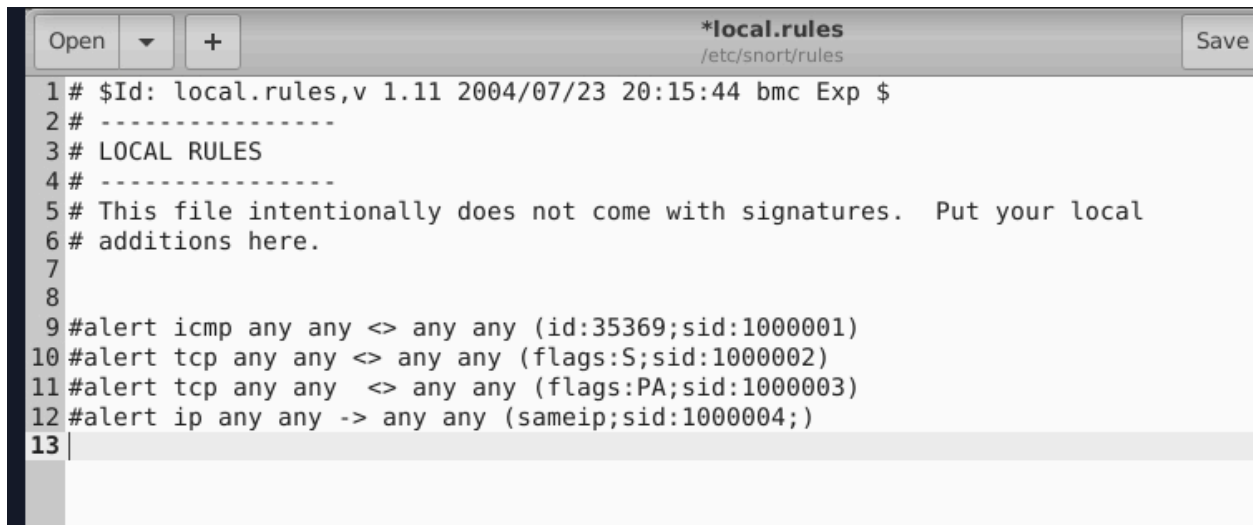
local.rules
/etc/snort/rules

```

1
2 # -----
3 # LOCAL RULES
4 # -----
5 # This file intentionally does not come with signatures. Put your local
6 # additions here.
7
8 alert icmp any any <> any any (msg: "ICMP Packet Found"; sid: 1000001; rev:1;)
9
10

```

created rule:



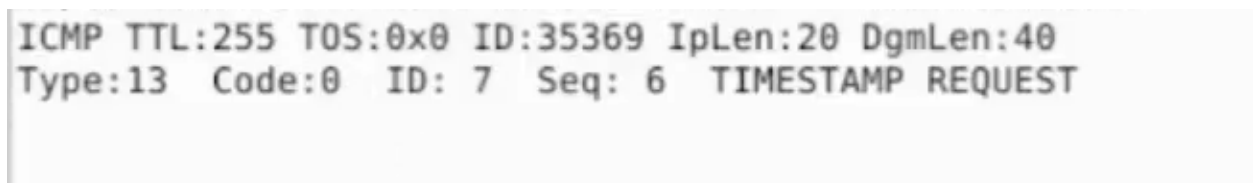
```
1 # $Id: local.rules,v 1.11 2004/07/23 20:15:44 bmc Exp $
2 # -----
3 # LOCAL RULES
4 # -----
5 # This file intentionally does not come with signatures.  Put your local
6 # additions here.
7
8
9 #alert icmp any any <> any any (id:35369;sid:1000001)
10 #alert tcp any any <> any any (flags:S;sid:1000002)
11 #alert tcp any any <> any any (flags:PA;sid:1000003)
12 #alert ip any any -> any any (sameip;sid:1000004;)
13
```

Answer the questions below

Use "task9.pcap". Write a rule to filter IP ID "35369" and run it against the given pcap file. What is the request name of the detected packet?

run the query: `"snort -c local.rules -A full -l . -r task9.pcap"`

TIMESTAMP REQUEST



```
ICMP TTL:255 TOS:0x0 ID:35369 IpLen:20 DgmLen:40
Type:13 Code:0 ID: 7 Seq: 6 TIMESTAMP REQUEST
```

Clear the previous alert file and comment out the old rules. Create a rule to filter packets with Syn flag and run it against the given pcap file. What is the number of detected packets? 1

Clear the previous alert file and comment out the old rules. Write a rule to filter packets with Push-Ack flags and run it against the given pcap file. What is the number of detected packets? 216

Clear the previous alert file and comment out the old rules. Create a rule to filter UDP packets with the same source and destination IP and run it against the

given pcap file. What is the number of packets that show the same source and destination address? 7

```
#alert icmp any any <> any any (id:35369;sid:1000001)
#alert tcp any any <> any any (flags:S;sid:1000002)
#alert tcp any any <> any any (flags:PA;sid:1000003)
alert ip any any -> any any (sameip; sid:1000004;)
```

Case Example - An analyst modified an existing rule successfully. Which rule option must the analyst change after the implementation? rev

Task 10 - Snort2 Operation Logic: Points to Remember

Main Components of Snort

- Packet Decoder - Packet collector component of Snort. It collects and prepares the packets for pre-processing.
- Pre-processors - A component that arranges and modifies the packets for the detection engine.
- Detection Engine - The primary component that processes, dissects, and analyzes the packets by applying the rules.
- Logging and Alerting - Log and alert generation component.
- Outputs and Plugins - Output integration modules (i.e., alerts to syslog/mysql) and additional plugins (rule management detection plugins) support is done with this component.

Snort Rules

- Community Rules - Free ruleset under the GPLv2. Publicly accessible, no need for registration.
- Registered Rules - Free ruleset (requires registration). This ruleset contains subscriber rules with 30 30-day delay.
- Subscriber Rules (Paid) - Paid ruleset (requires subscription). This ruleset is the main ruleset and is updated twice a week (Tuesdays and Thursdays).

Main configuration file (snort.conf). `sudo gedit /etc/snort/snort.conf`.

- Main configuration file.

- `local.rules`: User-generated rules file.

This section manages the scope of the detection and rule paths.

TAG NAME	INFO	EXAMPLE
HOME_NET	That is where we are protecting.	'any' OR '192.168.1.1/24'
EXTERNAL_NET	This field represents the external network, so we need to keep it set to 'any' or '!'\$HOME_NET'.	'any' OR '!'\$HOME_NET'
RULE_PATH	Hardcoded rule path.	/etc/snort/rules
SO_RULE_PATH	These rules are accompanied by registered and subscriber rules.	\$RULE_PATH/so_rules
PREPROC_RULE_PATH	These rules are accompanied by registered and subscriber rules.	\$RULE_PATH/plugin_rules

In this section, you manage the IPS mode of Snort. The single-node installation model IPS model works best with "afpacket" mode. You can enable this mode and run Snort in IPS.

TAG NAME	INFO	EXAMPLE
#config daq	IPS mode selection.	afpacket
#config daq_mode	Activating the inline mode	inline
#config logdir	Hardcoded default log path.	/var/logs/snort

Data Acquisition Modules (DAQs) are specialized libraries used for packet I/O, providing flexibility in processing packets. It is possible to select the DAQ type and mode for different purposes.

There are six DAQ modules available in Snort:

- PCAP: Default mode, known as Sniffer mode.
- Afpacket: Inline mode, known as IPS mode.
- Ipq: Inline mode on Linux by using Netfilter. It replaces the snort_inline patch.
- Nfq: Inline mode on Linux.
- Ipfw: Inline on OpenBSD and FreeBSD by using divert sockets, with the pf and ipfw firewalls.

- Dump: Testing mode of inline and normalisation.

The most popular modes are the default (pcap) and inline/IPS (Afpacket).

This section manages the outputs of IDS/IPS actions, including logging and alerting format details. The default action prompts everything in the console application, so configuring this part will help you use Snort more efficiently.

TAG NAME	INFO	EXAMPLE
# site-specific rules	Hardcoded local and user-generated rules path.	include \$RULE_PATH/local.rules
#include \$RULE_PATH/	Hardcoded default/downloaded rules path.	include \$RULE_PATH/rulename